

CSC 226 Assignment 1

An Nguyen, V01073373

22 Feb 2026

Part 1: Conjecture

For reasonable values of the load factor α , open addressing with linear probing provides better performance than chaining.

Investigation

To investigate the conjecture, I will analyze theoretically and experimentally the expected time complexity of both open addressing with linear probing and chaining for various load factors α . Then, I will compare the results to determine which method provides better performance.

1. Theoretical Analysis

In this part, I will leverage the theorems and corollaries from lectures and textbooks to (1) define the load factor α , (2) select a metric to compare the two methods, (3) derive the expected time complexity for both methods, and (4) compare the results to draw conclusions about the conjecture.

1.1. Define load factor α

In chaining, given a hash table T with m slots that stores n elements, we define the load factor α for T as n/m , that is, the average number of elements stored in a chain (α can be less than, or equal to, or greater than 1) [1].

In open addressing with linear probing, the load factor α is also defined as n/m , but it represents the proportion of occupied slots in the hash table. Note that at most one element occupies each slot, and thus $n \leq m$, which implies $\alpha \leq 1$. The analysis requires α to be strictly less than 1, and so we assume that at least one slot is empty [1].

1.2. Performance metric

I evaluate performance using two metrics. First, I analyze the expected number of probes, or the theoretical time complexity under the RAM model [1]. Second, I measure runtime in Python to determine whether the theoretical analysis holds in practice.

1.3. Expected time complexity

Theorem: For chaining, under the uniform hashing assumption (UHA), the expected running time of an INSERT operation or an unsuccessful FIND operation in a hash table using chaining to resolve collisions is $\theta(1 + \alpha)$, and the expected running time of a successful FIND operation is $\theta(1 + \alpha)$ [1].

Theorem: For open addressing with linear probing, under the UHA, the expected length of the probing sequence when linear probing is used to resolve collisions is approximately

$$\frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)} \right)$$

for a successful search, and approximately

$$\frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right)$$

for an unsuccessful search or an insertion [2].

1.4. Comparison

After computing, I have the following table and figures (graphs are plotted with R):

α	Chaining	Linear Probing (successful search)	Linear Probing (unsuccessful search or insertion)
0.2	1.2	1.125	1.281
0.25	1.25	1.167	1.389
0.3	1.3	1.214	1.52
0.35	1.35	1.269	1.683
0.4	1.4	1.333	1.889
0.45	1.45	1.409	2.153
0.5	1.50	1.500	2.500
0.55	1.55	1.611	2.969
0.6	1.60	1.750	3.625
0.65	1.65	1.929	4.582
0.7	1.70	2.167	6.056
0.75	1.75	2.500	8.500
0.8	1.80	3.000	13.000
0.85	1.85	3.833	22.722
0.9	1.90	5.500	50.500

Table 1: Expected runtime of search operations for chaining and linear probing.

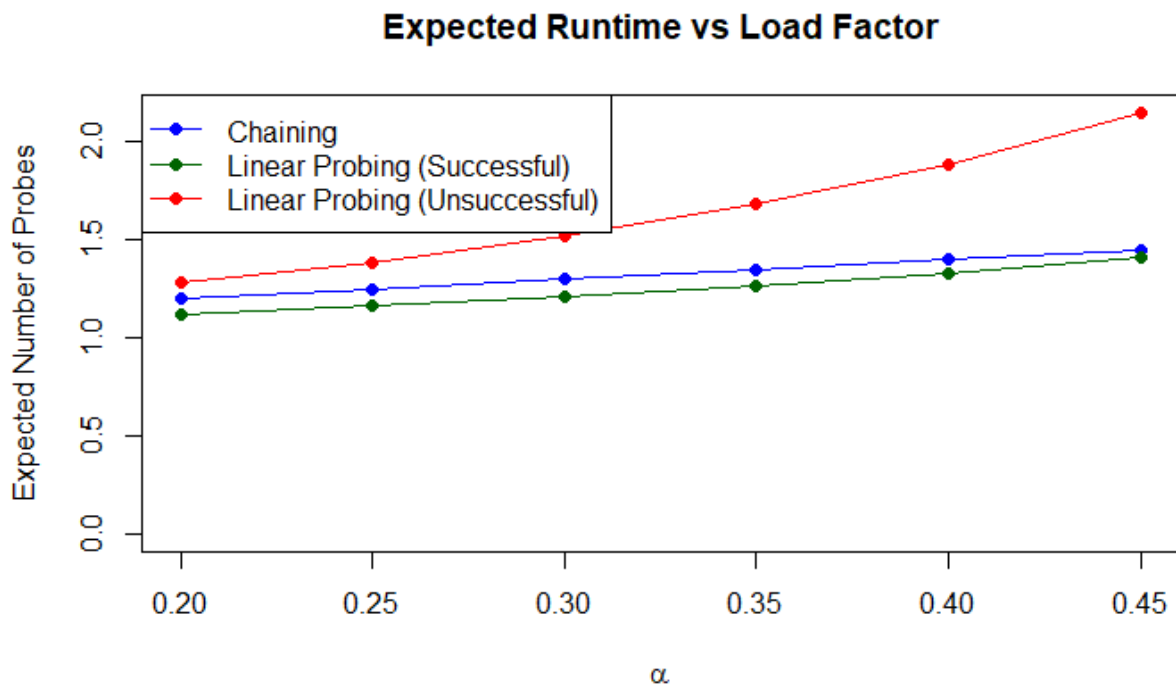


Figure 1: Expected number of probes with α from 0.2 to 0.45.

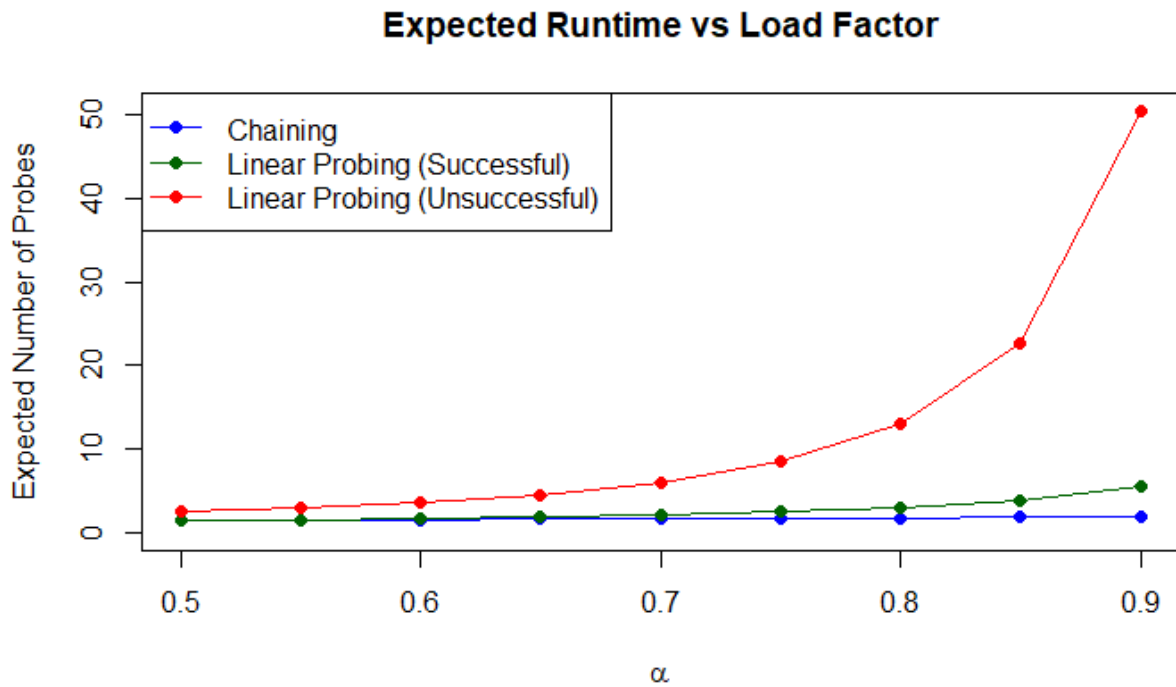


Figure 2: Expected number of probes with α from 0.5 to 0.9.

From the table and the first graph, it can be seen that for $\alpha \leq 0.5$, linear probing has a

slightly better expected time complexity than chaining for successful searches, and a worse expected time complexity for unsuccessful searches and insertions. However, as α increases beyond 0.5, the expected number of probes for unsuccessful searches and insertions in linear probing grows rapidly (due to clustering that will be explained later), while the expected cost of chaining only increases linearly.

The graph also shows that the red line begins to curve upward steeply around $\alpha = 0.7$, likely stemming from the primary clustering linear probing, in which runs of occupied slots build up, increasing the average search time [1]. In contrast, chaining does not suffer from clustering, as each slot can store multiple elements in a linked list, and thus the expected search time increases more gradually with α .

Reasonable values of α ?

The term "reasonable values of α " can be subjective. However, it is recommended keeping $\alpha < 0.5$ for linear probing to maintain performance, as beyond this point primary clustering begins to have a significant impact on probe lengths [3]. Upon the theoretical analysis, my initial conclusion is that the conjecture holds partially, as linear probing provides better performance than chaining for successful searches when $\alpha \leq 0.5$, but it does not provide better performance for unsuccessful searches and insertions, and its performance degrades significantly as α increases beyond recommended range.

Memory Hierarchy Considerations

The theoretical analysis above operates on the standard RAM model, in which all memory accesses are assumed to take constant time. However, in modern hardware, memory is organized in a hierarchy, with fast CPU caches at the top and slower main memory below. In this scenario, linear probing has a distinct advantage: probes are linear, and successive accesses tend to hit the same cache block, reducing the effective cost of each probe significantly. In the standard RAM model, primary clustering is a big issue for linear probing, as it can lead to long probing sequence. Yet, in practice, elements clustered together like this can actually improve cache performance, as the CPU can fetch multiple elements in a single cache line, minimizing the cache misses [1].

On the other hand chaining may have additional overhead due to reduced cache locality. Each slot in the hash table points to a linked list of elements, and accessing elements in the linked list may involve multiple cache misses, especially if the chains become long.

Indeed, Cormen et al. prove that if the hash function is 5-independent and $\alpha \leq 2/3$, the expected time for search, insertion, and deletion under linear probing is $O(1)$ [1]. This suggests that with a good hash function, linear probing can maintain constant expected time even at higher load factors, which may further support the conjecture in practice, despite the theoretical analysis under the RAM model.

2. Experimental Analysis

To further investigate the conjecture, I implement both chaining and linear probing in Python and run experiments to measure the actual time taken for search operations using different load factors α . For the purpose of this investigation, I will focus on the theoretical analysis under the RAM model, while acknowledging that real-world performance may differ due to memory hierarchy effects and the choice of hash function.

2.1. Implementation

I implement the chaining and linear probing hash tables in Python, ensuring that both implementations use the same hash function and handle collisions appropriately. I generate a set of random keys for insertion and unsuccessful search then measure the time taken for search operations under various load factors α .

2.2. Results and Comparison

After running the experiments, I collect the data on the time taken for search operations for both methods with multiple load factors to csv files. Gathered data can be presented in the following graphs (generated from the csv files using R):

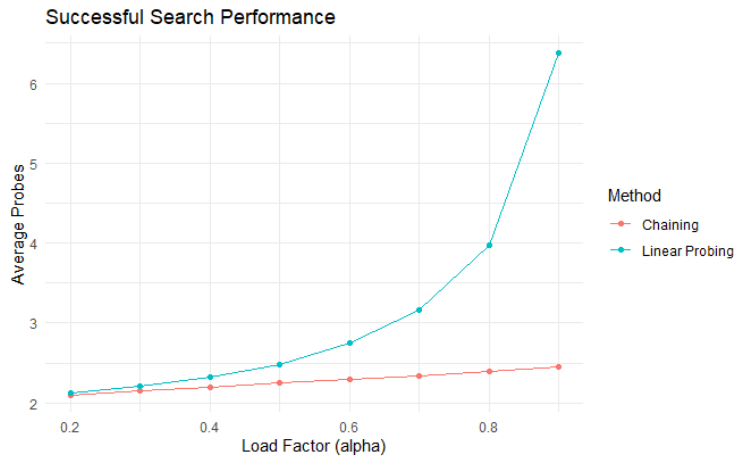


Figure 3: Average number of probes for successful search operations.

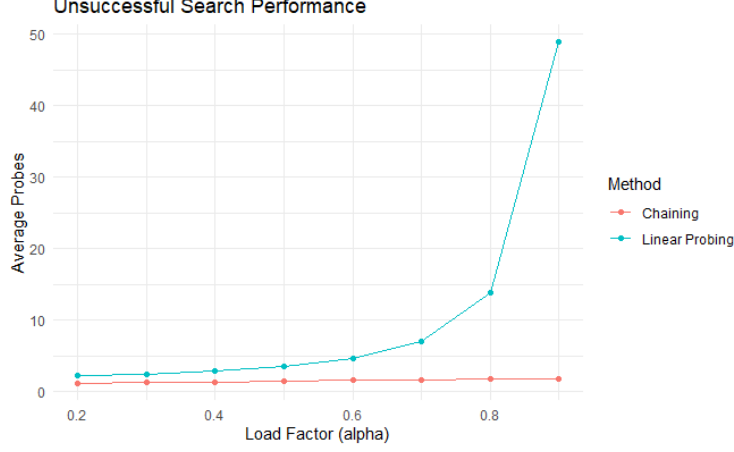


Figure 4: Average number of probes for unsuccessful search operations.

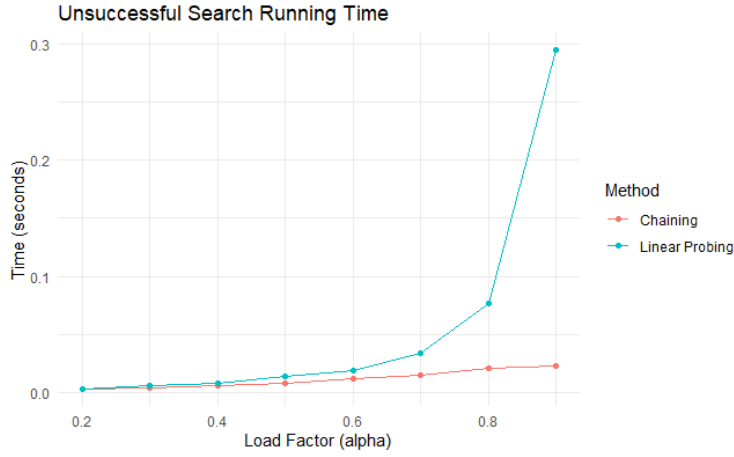


Figure 5: Runtime for unsuccessful search operations.

From the results above, the average number of probes for chaining grows linearly with α , consistent with the $\theta(1 + \alpha)$ expected time complexity. For linear probing, the average number of probes for successful searches remains relatively low for $\alpha \leq 0.5$, but it increases significantly as α approaches 1, particularly for unsuccessful searches. For runtime, we can see a similar trend, where the runtime for unsuccessful searches in linear probing increases dramatically as α increases, while the runtime for chaining increases more gradually, showing that number of probes can dominate performance.

Although the theoretical analysis suggests that linear probing may slightly outperform chaining for successful searches at very low load factors, this advantage was not observed in the experimental results (linear probing does not outperform chaining). The difference between the two methods at small values of α is minimal, and implementation-level factors such as Python overhead, memory allocation behavior, and cache effects may overshadow these small constant-factor differences. Thus, while the asymptotic analysis predicts a very slight advantage for linear probing under idealized assumptions, the practical impact appears negligible in this experimental setting.

Conclusion

Based on both the theoretical analysis and experimental results, my conclusion is the conjecture that linear probing provides better performance than chaining for reasonable values of the load factor does not hold in general. While linear probing may have a slight advantage for successful searches at very low load factors theoretically, this advantage is marginal and disappears quickly in practice as α increases. Furthermore, for unsuccessful searches and insertions, linear probing performs far worse than chaining, with performance degrading rapidly because of primary clustering when α approaches 1. The experiment results support these theoretical findings. In practice, linear probing did not outperform chaining for any tested load factor, and the steep growth in probe count for unsuccessful searches shows the instability of linear probing at higher load factors. Thus, the validity of the conjecture depends heavily on how “reasonable values” are defined. If α is below 0.5 and performance is evaluated primarily by successful searches, linear probing may be competitive. However, when considering all operations, chaining provides more stable and predictable performance. Therefore, the conjecture is not universally valid.

Limitations and Discussion

The analysis and experiments in this investigation are based on the assumption of uniform hashing, which may not always hold in practice. Additionally, the Knuth’s analysis of linear probing assumes an idealized model of hashing, and real-world performance may differ due to factors such as cache behavior and memory access patterns [1]. Linear probing in particular can be sensitive to the choice of hash function: poor hash functions can lead to increased clustering and even more degraded performance; on the other hand, sufficiently independent hash functions can help mitigate clustering and improve performance, even at higher load factors [1]. In my experiment, I only measure the average number of probes and runtime for search operations, which may not factor in Python’s overhead, garbage collector, and memory management, which can affect the actual performance of the implementations. Finally, taking into account the memory hierarchy considerations, linear probing may perform better than chaining in practice for lower load factors due to improved cache performance, even though it suffers from clustering in theory or in the standard RAM model, reasons as mentioned in the theoretical analysis.

Besides linear probing and chaining, alternative open addressing schemes such as quadratic probing and double hashing may have different performance characteristics. Quadratic probing reduces primary clustering by using non-linear probe sequences, though it can still suffer from secondary clustering, when hashing further resolves clustering with a second hash function for probe increments, generating probe sequences that are closer to uniform hashing. Therefore, double hashing can perform better at higher load factors compared to linear probing.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

- [2] D. E. Knuth, *The Art of Computer Programming, Vol. III*, 2nd ed., Addison-Wesley, 1998.
- [3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011.